KOÇ UNIVERSITY COLLEGE OF ENGINEERING

COMPUTER ENGINEERING DEPARTMENT

COMP 491 COMPUTER ENGINEERING DESIGN

PROJECT PROPOSAL

Fraud ML Engine for Bank Transaction Frauds

Bora Toker

Enes Tunç

Hilal Aygül

Merve Köybaşı

Project Advisor

Asst. Prof. Mehmet Emre Gürsoy



ABSTRACT:

Definition of Problem:

In today's world, with the drastic pace of improvement in technology, new fraud methods and techniques are emerging very frequently; rule-based detection systems can't catch up with rapidly evolving fraud techniques that are developed by cybercriminals. Moreover, rule-based fraud detection systems have a problem with a high number of false positive detections. Because of that, dynamic fraud detection has become an urgent need.

Motivation and Aim:

To address this, we are proposing a Fraud ML Engine for detecting fraudulent banking transactions. Due to strict banking policies, the requirement for high security, and a high number of legacy codes and architectures, our engine will have a hybrid architecture: a rule-based inference part that has static features, and an ML-based part that dynamically evaluates complex behavioral patterns. By combining deterministic business rules with adaptive AI capabilities, this hybrid approach ensures regulatory compliance while maximizing the accuracy of real-time anomaly detection, which allows detection of both textbook frauds and "innovative" frauds.

SYSTEM ARCHITECTURE AND METHODOLOGY

To build a robust, real-time fraud detection engine, this project employs a hybrid architecture combining deterministic rule-based evaluation with advanced machine learning algorithms. The methodology is divided into data extraction, hybrid risk scoring, and state-driven decision-making.

Feature Engineering (18-Dimensional Feature Vector):

The foundation of the ML Fraud Shield is its ability to extract contextual and behavioral data at the exact moment a transaction is initiated. The Feature Engineering Service generates an 18-dimensional feature vector for every transaction. These features capture various risk indicators, including:

- **Transaction and Amount Anomalies:** *Fraudsters typically attempt to siphon funds as quickly as possible. This category measures the financial abnormality of the transaction*.

    1. **Transfer Amount:** The monetary value of the transaction.
    2. **Transfer Magnitude:** The proportion of the value of the transaction to the value of the total account balance.
    3. **Rounds Amount:** Cybercriminals tend to transfer round amounts like 50000, 100000, etc.
    4. **Currency Mismatch:** Sudden transfers from a domestic currency account to a foreign currency account or an overseas crypto exchange are highly indicative of fraud.

- **Behavioral Profile:** *These features answer the question: "Does this transaction align with the user's historical habits?"*

    1. **30-Day Transfer Average:** If a user's historical average is 5,000 TL per month, a sudden 200,000 TL transfer creates a massive deviation, immediately increasing the ML risk score.
    2. **Night-time Transactions (00:00 – 06:00):** While legitimate users are typically inactive during these hours, cybercriminals prefer late-night windows to exploit sleeping victims and delayed bank interventions.
    3. **Daily Limit Utilization:** Fraudsters tend to maximize the daily transfer limit (e.g., utilizing 99% of it in a single transaction). Legitimate users rarely max out their daily limits.

- **Velocity and Temporal Features:** *These features detect automated bot attacks, panic behaviors, and impossible action speeds.*
    1. **Transaction Count in the Last 1 Hour:** A sudden spike in back-to-back transaction attempts indicates an automated script or a strategy to drain the account in smaller, less noticeable chunks.
    2. **Total Outflow in the Last 24 Hours:** Measures the cash outflow velocity.
    3. **Time Elapsed Since Last Transaction:** If a new transaction is initiated mere seconds after the previous one, it points to a bot rather than human reflexes.
    4. **Session Age:** If a massive transfer is initiated just seconds after a successful login, it is considered high risk. Genuine users typically spend time navigating menus and checking balances before initiating large transfers.

- **Recipient Trust & Social Network:** Analyzing *who is receiving the money is one of the most critical signals in fraud detection*.

    1. **First Transfer to Recipient:** Sending a large sum to a brand-new, never-before-seen IBAN significantly raises the risk score.

    2. **Past Transfer Count:** If the recipient is someone the user pays regularly), the ML model lowers the risk score, treating the recipient as "trusted."

    3. **Recipient in Saved Contacts:** A mathematical indicator of trust. Transfers made to pre-saved contacts drastically reduce the probability of the transaction being fraudulent.

- **Device & Identity Context:** *Even if a fraudster successfully steals a user's password, these hardware and location-based features act as a final barrier*.

    1. **Enrolled/Trusted Device Usage:** Checks whether the transaction is initiated from a cryptographically registered device or an entirely unrecognized computer.

    2. **Sudden Geo-Location Changes:** Detects if a user logs in from Istanbul in the morning and suddenly attempts a transfer from a Russian or US IP address two hours later, indicating VPN usage or a compromised account.

    3. **Failed Logins in the Last 24 Hours:** A high number of failed login attempts prior to a successful transaction is a clear footprint of a brute-force or credential-stuffing attack.

The Hybrid ML Architecture and Model Training:

Instead of relying solely on one method, the system processes the 18-feature vector through two parallel pipelines: a Rule-Based Engine and an ML-Based Engine. This dual approach was specifically chosen to ensure compliance with strict BDDK regulations and to provide a robust fallback mechanism, ensuring continuous fraud detection even if one of the engines experiences downtime.

Rule-based Engine: Evaluates static, strict banking compliance rules.

Machine Learning Engine (ONNX Runtime): The core intelligence of the system.

Data Partitioning:

To ensure rigorous evaluation and prevent data leakage, the highly imbalanced dataset of 10,000 synthetic banking transactions (9,500 legitimate and 500 fraudulent) was partitioned into three strictly isolated subsets: a Training Set (70%), a Validation Set (15%) for hyperparameter tuning, and a strictly isolated Hold-out Test Set (15%) for unbiased final evaluation.

Dual Algorithm Approach:

To capture both textbook frauds and novel attacks within this pipeline, a dual-algorithm approach was used:

Isolation Forest (Unsupervised): Applied to isolate anomalous data points and detect entirely new, previously unseen fraud patterns without relying on pre-existing labels.

XGBoost (Supervised): Trained and validated to classify known fraud topologies with high precision and optimal AUC-ROC metrics.

Explainable AI (XAI) via SHAP: Banking regulations strictly prohibit black-box AI systems where decisions cannot be audited or explained. To address this, the pipeline integrates SHAP. For every inference, SHAP calculates the exact percentage contribution of each of the 18 features (e.g., indicating that a transaction was blocked specifically because of a geographic anomaly and sudden velocity spike). This ensures that every AI-driven action is fully transparent and auditable by bank administrators.

ONNX Export and Ultra-Low Latency Integration: To bridge the gap between Python-based data science environments and enterprise-grade Java backends, the finalized ML model was exported in the ONNX (Open Neural Network Exchange) format. By embedding the ONNX Runtime directly into the Spring Boot microservice, the architecture entirely eliminates the network overhead of querying a separate Python microservice. This critical architectural decision enables the Fraud Inference Engine to evaluate transactions and generate a risk score with sub-5-millisecond latency, strictly meeting the real-time requirements of modern digital banking.

Transaction Evaluation:

Once the Machine Learning model is trained and deployed via ONNX, the system is ready for real-time inference. When a user initiates a transaction, the MlFraudInferenceEngine executes a strict, sub-5-millisecond evaluation process to determine the final security decision. This process operates in four sequential phases:

Phase 1: Real-Time Feature Extraction: The moment a transaction request hits the API Gateway, the FeatureEngineeringService intercepts it and queries the PostgreSQL database to construct the 18-dimensional feature vector.

Phase 2: Parallel Inference Execution: To balance deterministic security with AI adaptability, the feature vector is fed simultaneously into two parallel scoring engines:

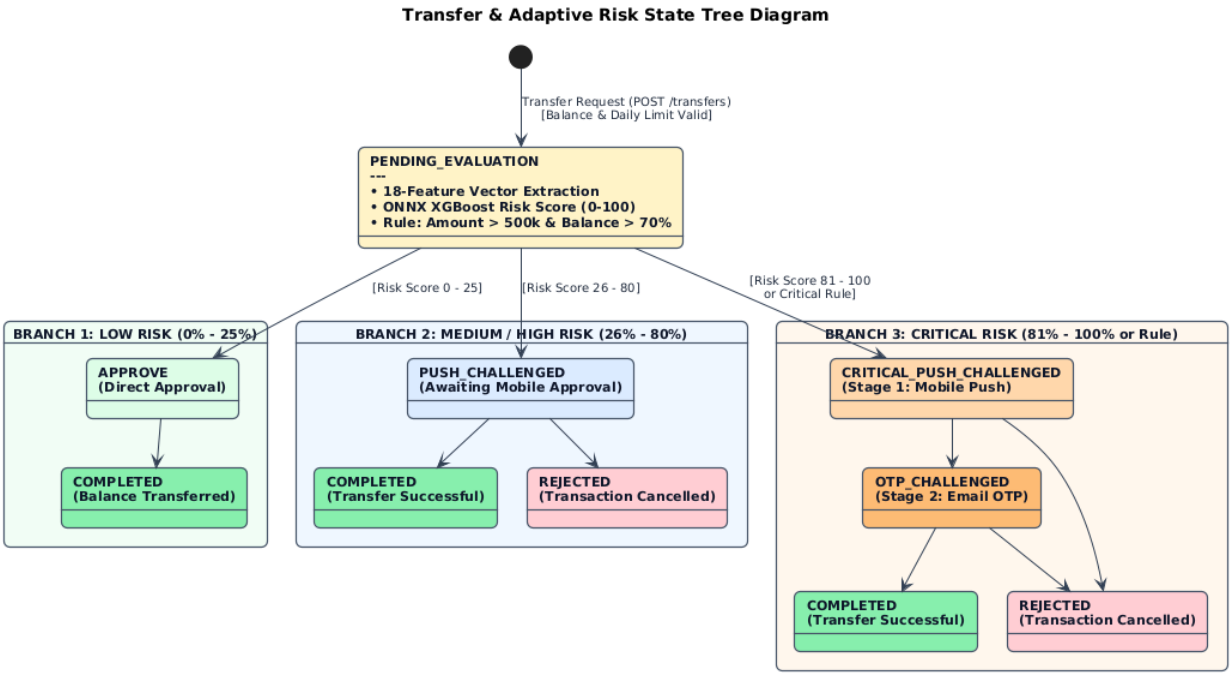
- **Rule-Based Inference:** Evaluates the vector against strict banking policies. For example, if a user attempts to transfer an amount exceeding their daily limit, or if the transaction originates from a blacklisted device, the rule-based score peaks immediately.

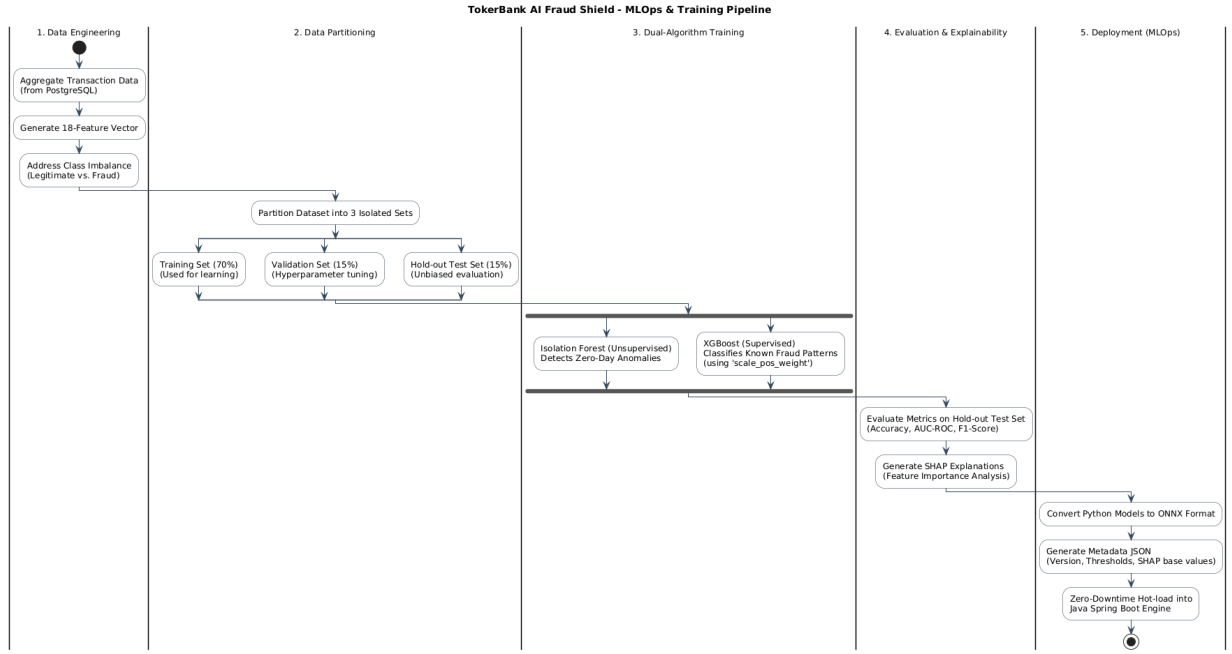
- **ML-Based Inference (ONNX):** The ONNX Runtime processes the exact same feature vector through the XGBoost algorithm, calculating the probability of the transaction being fraudulent based on behavioral anomalies (0 to 100).

Phase 3: Hybrid Score Calculation: To prevent a single engine from creating a bottleneck or false positive, the system fuses both outputs into a unified Final Risk Score. This is calculated using a weighted ensemble formula designed to prioritize behavioral intelligence while respecting strict rules:

Final Risk Score = (ML Probability Score × 0.7) + (Rule-based Score × 0.3)

Phase 4: State Tree Decision and Action Routing: Based on the computed Hybrid Risk Score, the transaction is dynamically routed through the Adaptive Risk State Tree, triggering one of three automated responses:





TokerBank AI Fraud Shield - Inference Flow

